

LEVEL 11 — Sorting and Searching with NumPy

Each topic below is shown line-by-line: code → output → explanation.

1. np.sort()

```
import numpy as np
a = np.array([5, 2, 8, 1, 4])
print(np.sort(a))
```

Output:
[1 2 4 5 8]

Explanation: np.sort() returns a sorted copy of the array. The original array is unchanged.

2. np.argsort()

```
a = np.array([50, 20, 80, 10])
idx = np.argsort(a)
print(idx)
print(a[idx])
```

Output:
[3 1 0 2]
[10 20 50 80]

Explanation: np.argsort() returns the indices that would sort the array.

3. np.searchsorted()

```
a = np.array([10, 20, 30, 40])
print(np.searchsorted(a, 25))
print(np.searchsorted(a, 30))
```

Output:
2
2

Explanation: It finds the position where a value should be inserted to keep a sorted array ordered.

4. np.partition()

```
a = np.array([9, 4, 7, 1, 6, 3])
print(np.partition(a, 2))
```

Output:
[1 3 4 7 6 9]

Explanation: The element at index 2 is in its final sorted position. Elements before it are smaller/equal and elements after it are larger/equal, but each side is not fully sorted.

5. np.argpartition()

```
a = np.array([9, 4, 7, 1, 6, 3])
idx = np.argpartition(a, 2)
print(idx)
print(a[idx])
```

Output:
[3 5 1 2 4 0]
[1 3 4 7 6 9]

Explanation: `np.argsort()` returns indices that produce the partitioned order.

6. `np.argmax()`

```
a = np.array([10, 50, 20, 40])
print(np.argmax(a))
print(a[np.argmax(a)])
```

Output:
1
50

Explanation: `np.argmax()` returns the index of the largest value.

7. `np.argmin()`

```
a = np.array([10, 50, 20, 40])
print(np.argmin(a))
print(a[np.argmin(a)])
```

Output:
0
10

Explanation: `np.argmin()` returns the index of the smallest value.

8. Sorting rows

```
a = np.array([
    [30, 10, 20],
    [60, 40, 50]
])

print(np.sort(a, axis=1))

Output:
[[10 20 30]
 [40 50 60]]
```

Explanation: `axis=1` sorts each row independently.

9. Sorting columns

```
a = np.array([
    [30, 60, 20],
    [10, 40, 50]
])

print(np.sort(a, axis=0))

Output:
[[10 40 20]
 [30 60 50]]
```

Explanation: `axis=0` sorts each column independently.

10. Descending sorting

```
a = np.array([5, 2, 8, 1, 4])
descending = np.sort(a)[::-1]
print(descending)
Output:
```

```
[8 5 4 2 1]
```

Explanation: NumPy sorts ascending by default. `[::-1]` reverses the sorted array to get descending order.

Quick Revision

<code>np.sort(a)</code>	-> sorted values
<code>np.argsort(a)</code>	-> sorting indices
<code>np.searchsorted(a, x)</code>	-> insertion position
<code>np.partition(a, k)</code>	-> partial sorting around index k
<code>np.argpartition(a, k)</code>	-> indices for partial sorting
<code>np.argmax(a)</code>	-> index of maximum
<code>np.argmin(a)</code>	-> index of minimum
<code>np.sort(a, axis=1)</code>	-> sort rows
<code>np.sort(a, axis=0)</code>	-> sort columns
<code>np.sort(a)[::-1]</code>	-> descending order

Important: axis=0 works down the columns; axis=1 works across the rows.